

Lista de 15 exercícios: escolher e resolver

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações | Unipampa

Como havia a possibilidade, pedi para a Claude AI gerar 15 exercícios de condicionais simples que focassem em Engenharia de Telecomunicações. Optei por resolver tudo em C pois é a linguagem que mais estou visualizando no momento, e quero me aprofundar em baixo nível.

1- Classificação de nível de sinal

Um receptor de rádio mede a potência do sinal recebido em dBm. Leia o valor e exiba: "Sinal excelente" se for maior que -50 dBm, "Sinal bom" se estiver entre -70 e -50 dBm, "Sinal fraco" se estiver entre -90 e -70 dBm, ou "Sem sinal" se for inferior a -90 dBm.

Resolução:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
float potencia_recebida = -20;
```

```
if(potencia_recebida >= -50)
```

```
{
```

```
printf("Sinal excelente");
```

```
}
```

```
else if( potencia_recebida > -70 && potencia_recebida < -50 )
```

```
{
```

```
printf("Sinal bom");
```

```
}
```

```
else if( potencia_recebida > -90 && potencia_recebida <= -70 )
```

```
{
```

```
printf("Sinal fraco");
```

```
}
```

```
else
```

```
{
```

```
printf("Sem sinal");
```

```
}
```

```
return 0;
```

```
}
```

2- Classificação de faixa de frequência

Dado um valor de frequência em MHz, classifique a faixa: "VHF" (30–300 MHz), "UHF" (300–3000 MHz), "SHF" (3000–30000 MHz), ou "Fora do espectro de interesse" para valores fora desses intervalos.

Resolução:

```
#include <stdio.h>
#include <string.h>

int main()
{
    float frequencia_mhz = 400;
    char faixa_frequencia[40];

    if (frequencia_mhz >= 30 && frequencia_mhz <= 300)
    {
        strcpy(faixa_frequencia, "VHF");
    }
    else if (frequencia_mhz >= 300 && frequencia_mhz <= 3000)
    {
        strcpy(faixa_frequencia, "UHF");
    }

    else if (frequencia_mhz >= 3000 && frequencia_mhz <= 30000)
    {
        strcpy(faixa_frequencia, "SHF");
    }

    else
    {
        strcpy(faixa_frequencia, "fora_espectro_interesse");
    }

    return 0;
}
```

3- Verificação de SNR mínima

Um sistema de comunicação exige SNR (relação sinal-ruído) mínima de 10 dB para operar. Leia o valor de SNR medido e informe se o enlace está "Operacional" ou "Abaixo do limiar — ajuste necessário".

Resolução:

```
#include <stdio.h>
```

```
int main()
{
```

```

float snr_medido = 20;

if(snr_medido >= 10)
{
printf("Operacional");
}
else
{
printf("SNR muito baixo");
}

return 0;
}

```

4- Atenuação no cabo

Uma fibra óptica possui atenuação de 0,35 dB/km. Leia o comprimento do enlace (em km) e a potência transmitida (em dBm). Calcule a potência recebida e verifique se ela é suficiente para o receptor, que exige no mínimo -20 dBm.

Resolução:

```

#include <stdio.h>

#define ATENUACAO 0.35

float calculo(float a, float b);

int main()
{
float comprimento_enlace = 20;
float potencia_transmitida = -30;
float potencia_recebida;

potencia_recebida = calculo (potencia_transmitida, comprimento_enlace);

if(potencia_recebida >= -20)
{
printf("É suficiente");
}
else
{
printf("Não é suficiente");
}

return 0;
}

```

```
float calculo(float a, float b)
{

return a - (ATENUACAO * b);
}
```

5- Seleção de modulação por SNR

Um sistema adaptativo escolhe a modulação com base na SNR disponível. Leia a SNR e exiba: "BPSK" (SNR < 10 dB), "QPSK" (10–15 dB), "16-QAM" (15–20 dB) ou "64-QAM" (acima de 20 dB).

Resolução:

```
#include <stdio.h>
#include <string.h>

int main()
{
float snr = 21;
char modulacao[10];

if(snr <= 10)
{
strcpy(modulacao, "BPSK");
}
else if(snr > 10 && snr <= 15)
{
strcpy(modulacao, "QPSK");
}
else if(snr > 15 && snr <= 20)
{
strcpy(modulacao, "16-QAM");
}
else
{
strcpy(modulacao, "64-QAM");
}

printf("Modulacao selecionada: %s\n", modulacao);

return 0;
}
```

6- Verificação de ganho de antena

Leia o ganho de uma antena em dBi e classifique: "Antena omnidirecional de baixo ganho" (≤ 3 dBi), "Antena direcional média" (4–15 dBi) ou "Antena de alto ganho" (> 15 dBi).

Resolução:

```
#include <stdio.h>
#include <string.h>

int main()
{
    float ganho= 5;
    char antena[50];

    if(ganho <= 3)
    {
        strcpy(antena, "Antena omnidirecional de baixo ganho");
    }
    else if(ganho >= 4 && ganho <= 15)
    {
        strcpy(antena, "Antena direcional média");
    }

    else
    {
        strcpy(antena, "Antena de alto ganho");
    }

    printf("Antena:: %s\n", antena);

    return 0;
}
```

7- Prioridade de tráfego (QoS)

Um roteador classifica pacotes por tipo de serviço. Leia o campo DSCP (0–63) e exiba: "Voz" (46), "Vídeo" (34), "Dados críticos" (26) ou "Tráfego de melhor esforço" para qualquer outro valor.

Resolução:

```
#include <stdio.h>

int main()
{
    int dscp = 26;

    if(dscp == 46)
    {
        printf("Voz");
    }
}
```

```

else if(dscp == 34)
{
printf("Vídeo");
}
else if(dscp ==26)
{
printf("Dados críticos");
}
else
{
printf("Tráfego de melhor esforço");
}
return 0;
}

```

8- Controle de potência de transmissão

Uma estação rádio-base ajusta sua potência de acordo com a distância ao terminal. Leia a distância (em metros) e exiba a potência recomendada: "10 mW" (≤ 100 m), "100 mW" (101–500 m), "500 mW" (501–2000 m) ou "Limite máximo — 1 W" (> 2000 m).

Resolução:

```

#include <stdio.h>

int main()
{

float metros = 350;

if (metros <= 100)
{
printf("Potência recomendada: 10mW");
}
else if (metros <= 500 && metros > 100)
{
printf("Potência recomendada: 100mW");
}
else if (metros <= 2000 && metros > 500)
{
printf("Potência recomendada: 500mW");
}
else
{
printf("Limite máximo: 1 W");
}
return 0;
}

```

9- Classificação de perda de percurso

A perda de percurso (path loss) de um enlace sem fio é medida em dB. Leia o valor e classifique: "Perda baixa" (menor que 80 dB), "Perda moderada" (80–110 dB), "Perda alta" (111–140 dB) ou "Perda crítica — enlace inviável" (acima de 140 dB).

Resolução:

```
#include <stdio.h>

int main()
{

float path_loss = 112;

if (path_loss < 80)
{
printf("Perda baixa");
}
else if (path_loss >= 80 && path_loss <= 110)
{
printf("Perda moderada");
}
else if (path_loss >= 110 && path_loss <= 140)
{
printf("Perda alta");
}
else
{
printf("Perda crítica");
}

return 0;
}
```

10- Verificação de endereço IP válido (octeto)

Leia um número inteiro representando um octeto de endereço IP. Verifique se está no intervalo válido (0 a 255). Se sim, exiba "Octeto válido"; caso contrário, exiba "Octeto inválido — fora do intervalo permitido".

Resolução:

```
#include <stdio.h>

int main()
{

int octeto_analisado = 224;
```

```

if(octeto_analisado >= 0 && octeto_analisado <= 255)
{
printf("Válido");
}
else
{
printf("Inválido");
}
return 0;
}

```

11- Detecção de interferência de canal

Dois canais Wi-Fi são considerados interferentes se a diferença entre seus números de canal for menor que 5 (para a faixa de 2,4 GHz). Leia os números de dois canais (1 a 13) e informe se há "Interferência entre canais" ou "Canais sem interferência".

Resolução:

```

#include <stdio.h>

int main()
{
int canal1 = 13;
int canal2 = 7;
int diferenca = 0;

diferenca = abs(canal1- canal2);  /*função que eu não conhecia! */

if(diferenca < 5)
{
printf("Dá interferência");
}
else
{
printf("Sem interferência");
}
return 0;
}

```

12- Classificação de tipo de fibra

Leia o diâmetro do núcleo de uma fibra óptica em micrômetros (μm). Classifique: "Fibra monomodo" (diâmetro entre 8 e 10 μm), "Fibra multimodo 50 μm " (diâmetro igual a 50) ou "Fibra multimodo 62,5 μm " (diâmetro igual a 62). Para outros valores, exiba "Tipo não reconhecido".

Resolução:

```

#include <stdio.h>

```



```

int main()
{
float diametro = 34;

if(diametro >= 8 && diametro <= 10)
{
printf("Fibra monomodo");
}
else if(diametro ==50)
{
printf("Fibra multimodo 50 micrômetros");
}
else if(diametro == 62)
{
printf("Fibra multimodo 62.5 micrômetros");
}
else
{
printf("Tipo não reconhecido");
}

return 0;
}

```

13- Conversão dBm para mW com classificação

Leia um valor de potência em dBm. Converta para mW usando a fórmula $P(\text{mW}) = 10^{(\text{dBm}/10)}$. Se o resultado for menor que 1 mW, exiba "Potência sub-miliWatt"; se estiver entre 1 e 1000 mW, exiba o valor em mW e "Potência em faixa operacional"; se for maior que 1000 mW, exiba "Potência elevada — verifique o equipamento".

Resolução:

```

#include <stdio.h>
#include <math.h>

int main()
{

float potencia_dbm = 60;

float mw = pow(10, (potencia_dbm/10));

if(mw < 1)
{
printf("Potência sub-miliWatt: %f", mw);
}

```

```

else if(mw >= 1 && mw <= 1000)
{
printf("Potência em faixa operacional: %f", mw);
}
else
{
printf("Potência elevada: %f", mw);
}

return 0;
}

```

14- Verificação de cobertura de célula

Uma célula de rede móvel tem raio máximo de 500 metros. Leia a distância (em metros) entre o terminal e a antena. Se estiver dentro do raio, exiba "Terminal dentro da célula"; se estiver entre 500 e 600 metros, exiba "Terminal na zona de borda — sinal instável"; caso contrário, exiba "Terminal fora de cobertura".

Resolução:

```

#include <stdio.h>

int main()
{
float raio = 340;

if(raio <= 500)
{
printf("Dentro da área");
}

else if(raio > 500 && raio <=600)
{
printf("Zona de borda");
}

else
{
printf("Fora de cobertura");
}

return 0;
}

```

15- Identificação de tipo de frame Ethernet

O campo EtherType de um frame identifica o protocolo encapsulado. Leia o valor já convertido hexadecimal (como inteiro): se for 0x0800 (2048), exiba "IPv4"; se for 0x86DD

(34525), exiba "IPv6"; se for 0x0806 (2054), exiba "ARP"; caso contrário, exiba "Protocolo desconhecido".

Resolução:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int EtherType = 2054;
```

```
if(EtherType == 2048)
```

```
{
```

```
printf("IPV4");
```

```
}
```

```
else if(EtherType == 34525)
```

```
{
```

```
printf("IPV6");
```

```
}
```

```
else if(EtherType == 2054)
```

```
{
```

```
printf("ARP");
```

```
}
```

```
else
```

```
{
```

```
printf("Protocolo desconhecido");
```

```
}
```

```
return 0;
```

```
}
```